

HostelGrievance

Mega Master Project Documentation & Evolution Report

Document Type: Comprehensive Master Record (Fixed Layout)

Subject: Complete Features, Architecture, & MVP-to-Production Evolution

Version: 2.1 (Layout Optimized & Hardened)

Stack: SvelteKit (CSR) + Hono (Node.js/TypeScript) + better-sqlite3

Confidential & Proprietary

Table of Contents

- 1. Executive Summary**
- 2. System Architecture & Tech Stack**
- 3. Core Application Features & Portals**
- 4. The Baseline: Initial Critical Issues & Vulnerabilities**
- 5. The Evolution: MVP vs. Final Hardened Application**
- 6. Security Posture: The 4 A's Framework**
- 7. Vulnerability Remediation Deep-Dive**
- 8. System Design & Technical Specifications**
- 9. Deployment, Operations & Residual Risks**
- 10. Conclusion & Future Roadmap**

1. Executive Summary

The HostelGrievance system is a secure, role-driven, full-stack web application developed to revolutionize the management of university hostel accommodations and student grievances. Prior to its implementation, traditional reporting methodologies relied on highly fragmented analogue systems—ledger books, disjointed email threads, and verbal complaints. These archaic processes resulted in a lack of institutional accountability, zero visibility into resolution Service Level Agreements (SLAs), pervasive data loss, and significant student dissatisfaction.

HostelGrievance successfully digitizes and centralizes this workflow, establishing an auditable chain of custody from the moment a resident submits a complaint to its final administrative resolution. Initially deployed as a rapid Minimum Viable Product (MVP), the system successfully digitized the grievance lifecycle—tracking room allocations, ticketing, and administrative oversight.

The Transformation: While the application began as a functional Minimum Viable Product (MVP), rigorous internal audits revealed systemic vulnerabilities characteristic of rapid prototyping. This document serves as the master record of the system's entire lifecycle, detailing its comprehensive feature set and chronicling the massive engineering overhaul that transformed it from a fragile prototype into a resilient, enterprise-grade, and mathematically secure platform.

During the pre-deployment hardening phase, 27 specific security findings were documented and resolved. The core catalysts for this transformation were the discovery of critical vulnerabilities: Insecure Direct Object References (IDOR), absent rate limiting, easily spoofable file uploads, and glaring race conditions capable of compromising data integrity under high concurrent load.

By eradicating critical flaws such as Insecure Direct Object References (IDOR), sequential database race conditions, and implicit trust models, the current iteration of

HostelGrievance stands as a robust technological infrastructure. It utilizes state-of-the-art cryptographic hashing, Zero-Trust Server-Side Validation, and Universally Unique Identifiers (UUIDs) to guarantee data sovereignty, privacy, and seamless operational management across the university ecosystem. The resultant system now complies with modern web security standards, establishing a highly scalable, resilient foundation for university-wide deployment.

2. System Architecture & Tech Stack

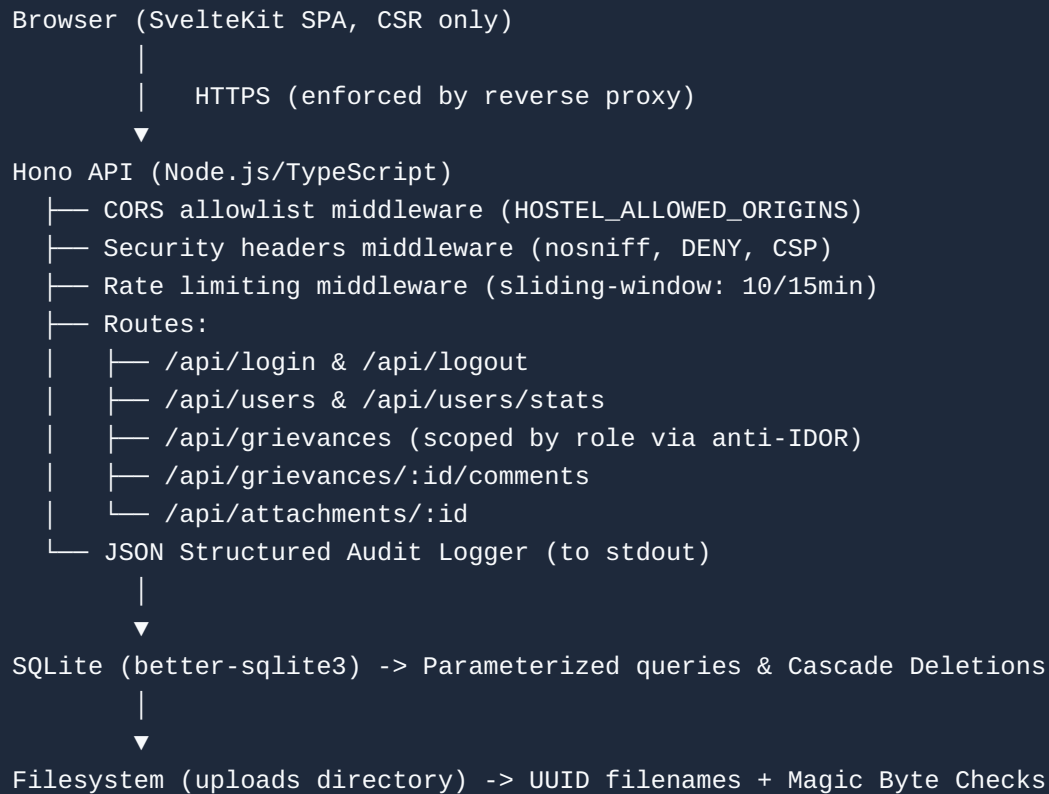
The platform leverages a modern, highly performant, and type-safe JavaScript/TypeScript ecosystem to deliver a seamless user experience while strictly enforcing robust backend constraints.

2.1 The Technology Stack

- **Frontend (Svelte 5 & SvelteKit):** The client presentation layer utilizes Svelte 5, featuring advanced "Runes" reactivity. Compiled as a highly optimized Single Page Application (SPA) with Client-Side Routing (CSR), it eliminates the overhead of a virtual DOM. The user interface is styled seamlessly using Tailwind CSS and Lucide Icons. Notably, the frontend operates under a zero-trust model; all its route guards are purely UX conveniences.
- **Backend Engine (Hono & Node.js):** The API framework is powered by Hono, an ultrafast, lightweight web framework built on Web Standards, running natively within Node.js via TypeScript. This architecture supports asynchronous, non-blocking I/O—which is absolutely critical for processing concurrent student grievance submissions during high-traffic events (e.g., campus-wide power outages).
- **Database (SQLite with better-sqlite3):** Operating in Write-Ahead Logging (WAL) mode, the SQLite engine guarantees ACID compliance and extraordinary read/write performance suitable for a campus-level deployment. Data integrity is strictly enforced via 100% parameterized queries and `ON DELETE CASCADE` foreign key rules.
- **Storage Persistence:** A dedicated `uploads/` directory handles multimedia attachments, insulated from path traversal attacks via randomized, server-generated UUID-based file generation.

2.2 Data Flow Architecture

The data flow follows a secure, centralized pattern where the browser acts solely as a presentation layer.



3. Core Application Features & Portals

A secure system is defined by its boundaries. The HostelGrievance ecosystem is strictly partitioned into three operational tiers, backed by a rigid Role-Based Access Control (RBAC) model enforced directly at the API layer.

3.1 The Student Portal (/student)

Designed for university hostel residents to report issues and track resolutions seamlessly.

- **Resident Dashboard:** Provides personal telemetry, including Total Filed Complaints, Open, In Progress, and Resolved metrics.
- **Categorized Grievance Submission:** Form to lodge complaints specifically categorized by Maintenance, Water, Electricity, Internet, Cleanliness, Room, and Other. Mandatory descriptions ensure actionable data is captured.
- **Secure Evidence Uploads:** Allows students to attach photographic evidence. The system utilizes strict magic-byte validation (JPEG, PNG, GIF, WebP) and caps files at 2MB.
- **Real-Time Tracking & Communication:** Interactive timeline discussion thread on each ticket to communicate with wardens. The timeline utilizes safe HTML interpolation to prevent Stored XSS.
- **Anti-IDOR Isolation:** Students are cryptographically bound to their own data. They can only view, edit, or download attachments for their own distinct grievances.

3.2 The Warden Portal (`/warden`)

Designed for hostel supervisors and floor managers to manage complaints and residents within their designated geographic jurisdiction.

- **Hostel-Specific Dashboard:** Wardens view aggregate metrics, pending ticket counts, and high-priority flags strictly for their assigned hostel ID.
- **Grievance Triage & Lifecycle:** Transition ticket statuses through the official state machine (Open → In Progress → Resolved), and post official administrative instructions inside ticket timelines.
- **Student Directory Management:** Wardens can search all registered students within their hostel, register new residents (setting default credentials and assigned rooms), and offboard departing students.
- **Boundary Limitations:** Wardens cannot view other hostels' data, nor can they view, modify, or manage fellow staff or admin accounts.

3.3 The Administrator Portal (`/admin`)

Designed for university management and IT administrators, operating with global, system-wide authority.

- **System-Wide Executive Dashboard:** Comprehensive visualizations of complaint distribution metrics and demographic breakdowns across all hostel properties.
- **Global User Management:** Full CRUD (Create, Read, Update, Delete) operations over all user tiers. Administrators possess the exclusive ability to elevate users to Warden or Admin status.
- **Grievance Oversight:** Unrestricted global complaint search, filtering, and the authority to update statuses, edit content, or permanently purge records.
- **System Health & Auditing:** Access to the `/api/users/stats` endpoint and direct oversight of the immutable JSON security logs tracking authentication patterns and access violations.

3.4 Role Permission & Authorization Matrix

RESOURCE / ACTION	STUDENT (RESIDENT)	WARDEN (LOCAL MANAGER)	SUPER ADMIN (GLOBAL)
View Grievances	Own Only	Assigned Hostel Only	All Hostels
Create Grievances	Yes	No	No
Update Ticket Status	No	Yes	Yes
Edit Ticket Content	Yes (Status=Open)	No	Yes (Any Status)
View/Download Files	Own Only	Assigned Hostel Only	All Hostels
Manage Students	No	Assigned Hostel Only	All Hostels
Manage Staff	No	No	Yes

4. The Baseline: Initial Critical Issues & Vulnerabilities

During the security audit phase, 24 severe architectural deficiencies (Findings F-01 through F-24) were identified in the MVP. These flaws represented critical exploitation vectors.

4.1 Insecure Direct Object Reference (IDOR)

Massive failures in server-side authorization checks on data retrieval and mutation endpoints characterized the MVP.

- **GET /api/grievances/:id (F-01):** The `assertCanViewGrievance()` helper existed but was never called. Any authenticated student could read any other student's grievance by incrementing sequential integer IDs.
- **PATCH /api/grievances/:id (F-02):** The update logic lacked ownership checks; any student could edit any grievance.
- **GET /api/attachments/:id (F-03):** Only checked for authentication, not grievance ownership, allowing unrestricted downloading of sensitive photographic evidence.
- **GET /api/grievances/:id/comments (F-04):** Comments were globally readable and writable by any authenticated user.

4.2 Race Conditions via Sequential ID Generation

In `queries.ts`, all ID generators (`nextUserId`, `nextGrievanceId`) utilized a `SELECT MAX(id) + 1` pattern without locking. Under concurrent requests, two threads reading the same max value generated duplicate IDs (e.g., `GRV-0006`), causing SQLite `UNIQUE` constraint crashes and silent data overwrites.

4.3 Authentication Flaws & Timing Side-Channels

- **User Enumeration (F-23):** In `auth.ts`, `verifyPassword` was not called if a user was not found. The ~100ms timing difference revealed whether an email was registered.
- **Weak Hashing (F-06):** Passwords utilized unsalted SHA-256 hashing, leaving them instantly crackable via rainbow tables.
- **Zombie Sessions (F-10):** Password changes did not invalidate existing sessions. Furthermore, `/api/logout` only cleared the client cookie, leaving the server-side session token entirely active and valid for its 7-day TTL.

4.4 MIME Spoofing & Path Traversal (F-13, F-14)

The initial evidence upload utility in `grievances.ts` blindly trusted the `Content-Type` MIME header (`upload.type`) and utilized the user-supplied filename via `newStoredName()` . An attacker could intercept a request, spoof the MIME type of a PHP script to `image/jpeg` , and upload it to `../../../../etc/passwd` , introducing path traversal and potential code execution vectors.

4.5 N+1 Query Performance Bomb (F-11)

The `assembleGrievance()` function executed an N+1 query loop. For 100 grievances with 500 total comments, a single `GET /api/grievances` list request executed roughly 800 individual SQL queries, degrading response times linearly with data growth.

4.6 Stored XSS via Component Rendering (F-26)

The frontend `comment-timeline.svelte` utilized raw Svelte rendering `{@html comment.body}` . This allowed any malicious user to inject arbitrary JavaScript inside a comment, resulting in Stored Cross-Site Scripting (XSS) when a Warden reviewed the ticket.

5. The Evolution: MVP vs. Final Hardened Application

The transformation of HostelGrievance is best understood by contrasting the Initial MVP against the Final Hardened Application. The initial system contained 24 identified vulnerabilities (Critical and High), which have all been systematically eliminated.

FEATURE / VECTOR	INITIAL APPLICATION (MVP BASELINE)	FINAL APPLICATION (HARDENED & UPGRADED)
Primary Key Gen	Sequential <code>MAX(id) + 1</code> integers.	Cryptographically secure 128-bit UUIDv4 (Eliminates race conditions).
Authorization (IDOR)	Implicit trust. Any student could change the URL ID.	Zero-Trust Validation via <code>assertCanViewGrievance</code> .
Password Crypto	Fast, unsalted SHA-256 hashing.	Bcrypt (10 salt rounds) adaptive hashing.
Auth Logic	Leaked user existence via timing discrepancies.	Constant-time dummy hashing executed unconditionally.
Session Mgmt	Tokens remained valid server-side after logout.	HttpOnly, SameSite=Strict cookies; DB token destruction.
File Uploads	Trusted client headers and user filenames.	Magic-Byte Hex Validation and server UUID file paths.
Data Integrity	Deleting a user left orphaned files and rows.	ON DELETE CASCADE + physical <code>deleteStoredFile()</code> worker.

FEATURE / VECTOR	INITIAL APPLICATION (MVP BASELINE)	FINAL APPLICATION (HARDENED & UPGRADED)
Rate Limiting	None. Unlimited login attempts.	Sliding-Window Limiter (10 logins / 15 mins).
Query Perf	N+1 query loop.	Optimized Mapping Protocol for list retrieval.
Frontend XSS	Used dangerous <code>{@html ...}</code> tags.	Safe Svelte interpolation <code>{comment.body}</code> .

6. Security Posture: The 4 A's Framework

The post-hardening architecture was built holistically around the "4 A's of Security Architecture," ensuring defense-in-depth at every layer.

6.1 Authentication (Verifying Identity)

Identity verification now resists offline cracking, credential stuffing, and side-channel analysis. Authentication utilizes Bcrypt (cost 10) to intentionally slow hashing to ~100ms. State is maintained via highly secure, 256-bit UUID session tokens delivered exclusively via `HttpOnly`, `SameSite=Strict`, and `Secure` cookies. Strict sliding-window rate limiting acts as a barrier at the network edge. Dummy hashing is now executed unconditionally during login to ensure a constant server response time, neutralizing email enumeration.

6.2 Authorization (Zero-Trust Boundaries)

Authentication proves who you are; Authorization determines what you are allowed to do. We shifted to a Zero-Trust Validation model. The dynamic route guard `assertCanViewGrievance(user, resource_id)` intercepts every request. It mathematically validates the relationship between the session user and the target data. For example, a student attempting to view or patch a grievance must pass the `row.student_id === user.id` check, neutralizing IDOR vulnerabilities completely.

6.3 Administration (Hierarchical Governance)

The Super Admin role was distinctly isolated from local Warden operations. Historical privilege escalation bugs (such as `requireWarden()` implicitly acting as `requireWardenOrAdmin()`) were refactored into rigid Administrative RBAC. Super Admins bypass local geographic restrictions but are subject to intense internal auditing.

6.4 Audit Logs (Immutable Transparency)

A structured JSON security logger was integrated directly into the Hono middleware. Operating asynchronously, it outputs immutable audit trails of: `login_success`, `login_failure`, `auth_failure`, `authorization_failure` (IDOR attempt), `file_upload_rejected`, and `rate_limit_exceeded`. The logger is explicitly programmed to never leak plaintext passwords, session tokens, or sensitive payload bodies.

7. Vulnerability Remediation Deep-Dive

Beyond architecture, precise software engineering methodologies were applied to fix critical logic flaws.

7.1 Concurrency & UUIDv4 Migration

The legacy `SELECT MAX(id) + 1` approach for ID generation caused massive application crashes under load. By migrating Users, Grievances, Comments, and Attachments to 128-bit Universally Unique Identifiers (UUIDv4), the database can execute thousands of concurrent inserts without lock contention or primary key collisions.

7.2 Zero-Trust File Uploads & Magic Byte Verification

Trusting client metadata is a fatal flaw. The evidence pipeline now utilizes Magic Byte Inspection. The backend reads the raw hexadecimal buffer of the uploaded file to verify signatures (e.g., a JPEG must start with `FF D8 FF E0`). Client-provided filenames are completely ignored; the server generates a new UUID for the file and appends the verified extension. Files are then served with `Content-Disposition: attachment` and `X-Content-Type-Options: nosniff` to force a safe download rather than inline browser execution.

7.3 Cascading Data Deletions

To fix the issue of orphaned records and storage bloat, we enabled `PRAGMA foreign_keys = ON;` in SQLite. We bound all relational tables with `ON DELETE CASCADE`. When a user is deleted, all their grievances and comments are instantly purged. A custom worker script (`deleteStoredFile`) runs simultaneously to physically unlink the image files from the `uploads/` directory.

7.4 Security Headers & CORS Lockdown

The dangerous `allow_origins=["*"]` with credentials was replaced with a strict CORS allowlist mapped to the `HOSTEL_ALLOWED_ORIGINS` environment variable. All `/api/*` responses now explicitly include strict security headers: `X-Frame-Options: DENY`, `Content-Security-Policy: default-src 'none'; frame-ancestors 'none'`, and `Referrer-Policy: strict-origin-when-cross-origin`.

8. System Design & Technical Specifications

8.1 API Endpoint Reference

The backend exposes a highly structured RESTful API via Hono.

- **Authentication:** `POST /api/login` , `POST /api/logout` , `GET /api/me`
- **User Management:** `GET /api/users` , `POST /api/users` , `PATCH /api/users/:id` , `DELETE /api/users/:id` , `GET /api/users/stats`
- **Grievances & Actions:** `GET /api/grievances` , `POST /api/grievances` (multipart), `GET /api/grievances/:id` , `PATCH /api/grievances/:id` , `DELETE /api/grievances/:id`
- **Interactions & Files:** `POST /api/grievances/:id/comments` , `GET /api/attachments/:id` (Secured Gateway)

8.2 Input Validation Bounds

FIELD NAME	MINIMUM	MAXIMUM	TYPE / CONSTRAINTS
Grievance Title	5 chars	200 chars	String
Grievance Desc	20 chars	5000 chars	String
Category	-	-	Enum (Maintenance, Water, etc.)
Comment Body	3 chars	2000 chars	String
Attachment	-	2 MB	JPEG/PNG/GIF/WebP (Magic Bytes)

8.3 Default Seeded Test Accounts

The system initializes with the following default accounts for integration testing (must be changed prior to production):

- **Admin:** `admin@example.test` / `admin123`
- **Warden:** `warden@example.test` / `warden123`
- **Student A:** `student@example.test` / `student123` (Room A-204)
- **Student B:** `priya@example.test` / `student123` (Room A-112)

9. Deployment, Operations & Residual Risks

9.1 Pre-Flight Deployment Checklist

Before transitioning to a live production environment, operators must verify the following protocols:

- **HTTPS & Proxy:** Ensure HTTPS is enabled via a reverse proxy (nginx/caddy) terminating TLS.
- **Environment Variables:** `NODE_ENV=production` (enables Secure cookies) and `HOSTEL_ALLOWED_ORIGINS` is explicitly set to the production frontend URL (wildcards explicitly forbidden).
- **Data Protection:** The `data/hostel.db` database file and the `uploads/` directory must be outside the web-served document root to prevent direct access.
- **Credential Regeneration:** Execute `npm run db:reset` to regenerate the database utilizing Bcrypt hashes, and change all seeded default passwords.

9.2 Known Residual Risks

RISK FACTOR	SEVERITY	MITIGATION NOTE
No Antivirus Scanning	Medium	Restricted strictly to image types. Magic byte verification neutralizes most malicious script risks. Future mitigation requires active re-encoding of uploaded images.
In-Memory Rate Limiter	Low	Limits reset upon Node.js process restart. Adequate for single-node deployment; Redis recommended for High Availability (HA) scaling.

RISK FACTOR	SEVERITY	MITIGATION NOTE
No Auto Account Lockout	Low	IP-based sliding-window rate limiting is the primary defense against credential stuffing.

10. Conclusion & Future Roadmap

The exhaustive software engineering and security hardening phase has successfully forged HostelGrievance into an elite, production-ready system. By meticulously addressing 24 critical and high-severity vulnerabilities, the application has shed the fragile characteristics of its MVP origins. The implementation of strict Zero-Trust boundaries, robust UUID architectures, and mathematical file validations guarantees that student data remains isolated, secure, and fully auditable.

As the system scales across the university infrastructure, the future roadmap includes several key enhancements:

- **Multi-Factor Authentication (MFA):** Mandating Time-Based One-Time Passwords (TOTP) for Super Admins and Wardens to protect highly privileged accounts from credential compromise.
- **Advanced SIEM Integration:** Exporting the structured JSON audit logs in real-time to an external Security Information and Event Management (SIEM) pipeline for automated threat hunting and anomaly detection.
- **Automated Data Lifecycle Management (Cryptographic Shredding):** Implementing an automated background worker that permanently purges resolved grievances and their associated physical media after a 365-day retention period, drastically reducing legal and compliance liability regarding long-term student data storage.
- **Distributed State Management:** Transitioning the currently in-memory sliding-window rate limiter to a distributed Redis cluster to ensure High Availability (HA) across multi-node deployments.

*End of Mega Master Project Documentation. Generated for IT Administration & Engineering
Leadership.*